Topology Catalog Module Specification

Normative Multi-Frame Framework Topology API for mdstats Stage 3

mdstats

2026-07-14 (implemented Stage 3 revision)

Contents

Purpose and status

This document specifies the implemented public module

`mdstats/analysis/topology_catalog.py`

for mdstats Stage 3. The API and algorithms described here are implemented in `mdstats 0.16.0`. This document is the normative contract for maintenance, review, AI contextualization, and later primitive-ring integration.

The module converts a multi-frame atomic-connectivity result into a compressed catalog of exact projected framework topologies. It requires the repaired Stage 2 schemas `mdstats.framework-mapping.v2` and `mdstats.framework-topology.v2`, where projected adjacency is undirected but each edge retains an orientation-aware ordered atomic path. It answers four distinct questions:

1. Which unique framework topologies occur?
2. Which topology belongs to each selected frame?
3. For a trajectory, where are the contiguous topology segments and transitions?
4. Which atomic and projected framework edges changed at each transition?

The central transformation is

$$\texttt{AtomicConnectivityResult} \xrightarrow{\text{fixed FrameworkMapping}} \texttt{TopologyCatalog}.$$

The module must project each unique atomic-connectivity state at most once in catalog mode. It must then deduplicate projected framework topologies by exact canonical equality, not by frame order, geometric similarity, or digest equality alone.

This module does not enumerate rings. Its output is the authoritative multi-frame topology input for `primitive_ring.py`.

Design motives

A long trajectory may contain many frames but only a few connectivity states. In addition, several atomic-connectivity states may project to the same framework topology. For example, spectator Na-O contacts may change while the Si-Al framework graph remains identical.

Rebuilding and storing one framework graph per frame would therefore:

- repeat expensive linker-path projection;
- duplicate identical graph objects;
- conflate atomic-connectivity changes with framework-topology changes;

- obscure recurring topology classes such as $A \to B \to A$;
- encourage accidental temporal interpretation of unordered ensembles;
- make later ring search unnecessarily expensive.

Stage 3 introduces an explicit compression and classification layer:

```
selected frames
      |
      v
atomic connectivity state IDs
      |
      v
project each unique state once
      |
      v
exact framework topology classes
      |\
      | +--> unordered frame groups
      |
      +----> ordered trajectory segments and transitions
```

The scientific separation is

$$\boxed{\text{frame semantics} \neq \text{atomic connectivity state} \neq \text{framework topology class}}.$$

A trajectory is time ordered, but it may be uniform, partitioned, or intentionally per-frame. An ensemble is unordered, but it may contain one or several exact topology classes.

Responsibility boundaries

Owned by `topology_catalog.py`

The module owns:

- validation of compatibility among the collection, connectivity result, and framework mapping;
- projection of unique atomic-connectivity states;
- exact deduplication of projected framework topologies;
- deterministic topology IDs;
- frame-to-topology assignment;
- topology consistency classification;
- frame groups for every topology class;
- contiguous segments for trajectories;
- exact transition records at trajectory segment boundaries;
- atomic-edge and projected-edge differences;
- descriptive persistence labels for short trajectory segments;
- stable serialization and catalog digests;
- complete provenance for the classification policy.

Not owned by this module

The module must not:

- infer atomic connectivity from coordinates;
- modify connectivity scope or connectivity definitions;
- smooth or repair atomic-connectivity states;
- change framework roles or linker-path rules between states;
- compare structures with different atom ordering without an explicit identity mapping;
- infer temporal transitions for ensembles;
- merge short-lived topology segments silently;
- enumerate primitive rings;

- infer approximate ring lineage;
- calculate ring geometry, sites, or cages;
- classify dynamic spatial regions;
- use plotting objects as topology authority.

The dependency direction is

```
AtomisticFrameCollection
          |
          +--> AtomicConnectivityResult
          |           |
          |           v
          +--> FrameworkMapping
                      |
                      v
              TopologyCatalog
                      |
                      +--> primitive_ring.py
                      +--> transition analysis
                      +--> framework visualization
```

Terminology and normative language

- **Selected frame position**: zero-based position inside the analyzed result, distinct from the source collection frame index.
- **Connectivity state**: one immutable `AtomicConnectivityState` stored in an `AtomicConnectivityResult`.
- **Projected topology**: one `FrameworkTopology` obtained by applying one fixed `FrameworkMapping` to one connectivity state.
- **Topology class**: an exact equivalence class of projected framework topologies.
- **Frame group**: all selected frame positions assigned to one topology class. It has no temporal meaning.
- **Trajectory segment**: one maximal contiguous run of a topology ID in selected trajectory order.
- **Transition**: the exact boundary between two adjacent trajectory segments.
- **Transient segment**: a segment shorter than a configured descriptive persistence threshold. It remains present in the result.
- **Catalog mode**: exact class reconciliation and compressed topology storage.
- **Per-frame mode**: one public topology record per selected frame, with no cross-frame identity promise.

The words **must**, **must not**, **should**, and **may** are normative.

Stage 2 edge-identity dependency

The catalog does not reinterpret projected edges. It consumes the exact Stage 2 identity contract:

$$\text{undirected adjacency} + \text{orientation-aware path decoration}\ .$$

For a bridge

$$P_{uv} = (u, \ell_1, ..., \ell_m, v),$$

the reverse traversal

$$P_{vu} = (v, \ell_m, ..., \ell_1, u)$$

is the same physical edge. Thus `A-O-S-B` and `B-S-O-A` belong to one canonical edge class. `A-O-S-B` and `A-S-O-B` are distinct decorated edges.

Topology reconciliation must compare the canonical Stage 2 edge records. It must not independently sort endpoints and linker species, simplify the graph to bare endpoint pairs, or convert reverse traversal into a second directed edge.

Structural identity versus path provenance

Stage 2 stores both a structural edge key and richer path provenance:

```
FrameworkEdgeKey   -> canonical structural edge identity
FrameworkEdgePath  -> structural key plus reconstruction and diagnostic provenance
OrientedFrameworkEdgePath -> derived traversal view
```

Stage 3 topology classes must use the Stage 2 **structural identity contract**, not object equality over every provenance field. For one topology T, define the canonical structural key

$$K(T) = (\nu_T, D_M, \mathbf{p}, V_T, Z_T, E_T),$$

where

- ν_T is the framework-topology schema version;
- D_M is the fixed mapping digest;
- $\mathbf{p}$ is the periodic-dimensionality flag tuple;
- V_T and Z_T are the aligned canonical vertex indices and atomic numbers;
- E_T is the sorted tuple of canonical `FrameworkEdgeKey` records.

The following are provenance or derived diagnostics and must not independently split topology classes when $K(T)$ is unchanged:

- `source_connectivity_digest` except as representative provenance;
- `projection_report`;
- `validation` findings;
- `raw_image_shift` before projected-gauge normalization;
- oriented $+1/-1$ traversal views;
- degree and component arrays, which are deterministically derived from V_T and E_T and must instead be validated for consistency.

`topology.graph_digest` and `topology.digest` are deterministic bucket keys. A digest match must still be followed by exact comparison of $K(T)$.

This distinction prevents two projections of the same undirected decorated graph from being separated merely because they were discovered in opposite traversal orders or carry different nonstructural diagnostics.

Mathematical model

Let the selected frames be indexed by result position

$$p \in \{0, 1, ..., F-1\}.$$

Let

$$c_p \in \{0, 1, ..., S-1\}$$

be the atomic-connectivity state ID assigned to selected position p.

For a fixed mapping M, projection defines

$$P_M : C_s \mapsto T_s,$$

where C_s is connectivity state s and T_s is its projected framework topology.

Two connectivity states are topology-equivalent under M when

$$C_a \underset{M}{\sim} C_b \Leftrightarrow P_M(C_a) = P_M(C_b),$$

where equality is exact equality of canonical decorated framework records.

The topology catalog stores the quotient classes

$$\{C_s\}/\underset{M}{\sim}.$$

If q_s maps connectivity state s to topology ID, then the frame topology ID is

$$t_p = q_{c_p}.$$

Digest equality is only a lookup accelerator. The authoritative relation is structured equality:

same digest $\not\Rightarrow$ same topology without exact comparison .

Public API overview

The module exports the following symbols from `mdstats.analysis` and re-exports them from `mdstats`:

```
CANONICAL_TOPOLOGY_CATALOG_SCHEMA
TOPOLOGY_CATALOG_DIGEST_ALGORITHM

TopologyConsistency
TopologySegmentStatus
TopologyCatalogOptions
TopologyFrameGroup
TopologySegment
TopologyTransition
TopologyCatalog

build_topology_catalog
```

The public exceptions are

```
TopologyCatalogError
TopologyCatalogInputError
TopologyCatalogProjectionError
TopologyCatalogConsistencyError
TopologyCatalogSerializationError
```

All identity-bearing public objects must be immutable after construction. NumPy arrays stored by result objects must be copied into canonical dtypes and marked read-only.

Constants and schemas

```
CANONICAL_TOPOLOGY_CATALOG_SCHEMA = "mdstats.topology-catalog.v1"
TOPOLOGY_CATALOG_DIGEST_ALGORITHM = "sha256"
```

The schema version must be included in serialized payloads and the catalog digest. A schema change that alters canonical equality, topology ID assignment, transition fields, or serialization order requires a new schema string.

Topology consistency

```
from enum import Enum


class TopologyConsistency(str, Enum):
    UNDEFINED = "undefined"
    UNIFORM = "uniform"
    PARTITIONED = "partitioned"
    PER_FRAME = "per_frame"
```

The meanings are:

- `UNDEFINED`: no topology classification has been computed. This value is reserved for higher-level placeholders and must not be returned by `build_topology_catalog()`.
- `UNIFORM`: catalog mode found exactly one exact topology class.
- `PARTITIONED`: catalog mode found two or more exact topology classes.
- `PER_FRAME`: the caller requested independent per-frame topology records.

The category is descriptive. It says nothing about whether frames are time ordered.

Segment persistence status

```
class TopologySegmentStatus(str, Enum):
    CONFIRMED = "confirmed"
    TRANSIENT = "transient"
```

For a trajectory segment of length

$$L_k = b_k - a_k,$$

and configured minimum persistence m,

$$\text{status}(k) = \begin{cases} \texttt{CONFIRMED}, L_k \geq m, \\ \texttt{TRANSIENT}, L_k < m. \end{cases}$$

This status is descriptive only. The module must not delete, merge, relabel, or repair a transient segment. Exact frame assignments and transitions remain available.

For ensembles, segment status is not defined because segments are not created.

Catalog options

```
from dataclasses import dataclass
from typing import Literal


@dataclass(frozen=True, slots=True)
class TopologyCatalogOptions:
    mode: Literal["catalog", "per_frame"] = "catalog"
    minimum_persistent_frames: int = 1
    include_atomic_edge_differences: bool = True
    include_framework_edge_differences: bool = True
```

Field meanings

`mode`

`"catalog"` performs exact class reconciliation and compressed storage.

`"per_frame"` creates one public `FrameworkTopology` entry per selected frame and sets

$$\texttt{frame\_topology\_ids}[p] = p.$$

The implementation may memoize projection internally for repeated connectivity states, but it must not expose persistent cross-frame topology identity in per-frame mode.

`minimum_persistent_frames`

Positive integer used only to label trajectory segments as confirmed or transient. The default value 1 confirms every nonempty segment.

The value must not affect frame-to-topology assignment.

Difference flags

The two `include_*_differences` flags control storage of transition edge lists. Transition boundaries, topology IDs, and affected frame indices must still be recorded when detailed edge lists are disabled.

Disabling differences is a memory policy, not a change in topology classification.

Option constraints

- `mode` must be `"catalog"` or `"per_frame"`.
- `minimum_persistent_frames` must be a positive integer.
- Boolean fields must be actual booleans, not arbitrary truthy objects.
- Options must be serializable and included in provenance.

Frame groups

```
@dataclass(frozen=True, slots=True)
class TopologyFrameGroup:
```

```
    topology_id: int
    result_positions: NDArray[np.int64]
```

A frame group stores every selected result position assigned to one topology ID.

Constraints

- `topology_id` is nonnegative and valid for the owning catalog.
- `result_positions` is one-dimensional, nonempty, strictly increasing, unique, and read-only.
- Groups are sorted by `topology_id`.
- Groups form an exact partition of all selected result positions.
- Group order does not imply temporal order.

Frame groups are created for both trajectories and ensembles. They provide the semantics-neutral class membership representation.

Trajectory segments

```
@dataclass(frozen=True, slots=True)
class TopologySegment:
    segment_id: int
    topology_id: int
    result_position_start: int
    result_position_stop: int
    status: TopologySegmentStatus
```

The interval is half-open:

$$[a_k, b_k).$$

A segment is maximal, meaning

$$t_p = t_{a_k} \quad \text{for all} \quad p \in [a_k, b_k),$$

and adjacent selected positions outside the interval, when present, have a different topology ID.

Constraints

- Segment IDs are dense and begin at zero.
- Intervals are nonempty and contiguous in result-position space.
- Segments cover all selected positions exactly once.
- Adjacent segments have different topology IDs.
- `status` follows `minimum_persistent_frames` exactly.
- Segments exist only when the collection has trajectory semantics and mode is `"catalog"`.
- Ensemble catalogs and per-frame catalogs store `segments=None`.

Topology transitions

```
@dataclass(frozen=True, slots=True)
class TopologyTransition:
    transition_id: int
    source_segment_id: int
    target_segment_id: int
    source_topology_id: int
    target_topology_id: int
    source_connectivity_state_id: int
    target_connectivity_state_id: int
    result_position_before: int
    result_position_after: int
    collection_frame_index_before: int
    collection_frame_index_after: int
    frame_id_before: int
    frame_id_after: int
```

```
    added_atomic_edges: tuple[AtomicEdgeKey, ...]
    removed_atomic_edges: tuple[AtomicEdgeKey, ...]
    added_framework_edges: tuple[FrameworkEdgeKey, ...]
    removed_framework_edges: tuple[FrameworkEdgeKey, ...]
    affected_atom_indices: tuple[int, ...]
    affected_vertex_atom_indices: tuple[int, ...]
    affected_linker_atom_indices: tuple[int, ...]
```

A transition exists at every boundary between adjacent topology segments.

For source state A and target state B, atomic differences are

$$\Delta E^{+}_{\text{atomic}} = E_{\text{atomic}}(B) \setminus E_{\text{atomic}}(A),$$

$$\Delta E^{-}_{\text{atomic}} = E_{\text{atomic}}(A) \setminus E_{\text{atomic}}(B).$$

For source topology T_A and target topology T_B, framework differences are

$$\Delta E^{+}_{\text{framework}} = E(T_B) \setminus E(T_A),$$

$$\Delta E^{-}_{\text{framework}} = E(T_A) \setminus E(T_B).$$

The affected atom set is the sorted union of endpoints of changed atomic edges and all atom indices appearing in changed framework paths. The affected vertex set contains changed projected-edge endpoints. The affected linker set contains internal linker atoms in changed projected edges.

Transition constraints

- Transitions exist only for trajectory catalog mode.
- Transition IDs are dense and follow segment-boundary order.
- `result_position_after = result_position_before + 1`.
- Source and target segment IDs are adjacent.
- Source and target topology IDs differ.
- Source and target connectivity state IDs are the frame state IDs immediately before and after the boundary.
- Edge tuples are canonical, deduplicated, and lexicographically sorted.
- Empty atomic-edge differences are valid when different connectivity states are not the cause of the topology boundary only if an externally constructed result is being validated; normal Stage 3 construction should preserve exact source evidence.
- Empty framework-edge differences are invalid for a topology-changing boundary.
- A transition into or out of a transient segment remains recorded.

For ensembles, `transitions=()` regardless of stored frame order.

Topology catalog result

```
@dataclass(frozen=True, slots=True, eq=False)
class TopologyCatalog:
    mapping: FrameworkMapping
    validation_rules: FrameworkValidationRules | None
    projection_options: FrameworkProjectionOptions
    catalog_options: TopologyCatalogOptions
    frame_semantics: FrameSemantics
    consistency: TopologyConsistency

    frame_indices: NDArray[np.int64]
    frame_ids: NDArray[np.int64]
    frame_connectivity_state_ids: NDArray[np.int32]
    connectivity_state_topology_ids: NDArray[np.int32]
    frame_topology_ids: NDArray[np.int32]

    topologies: tuple[FrameworkTopology, ...]
    frame_groups: tuple[TopologyFrameGroup, ...]
    segments: tuple[TopologySegment, ...] | None
```

```
    transitions: tuple[TopologyTransition, ...]

    metadata: Mapping[str, Any]
    canonical_schema_version: str = CANONICAL_TOPOLOGY_CATALOG_SCHEMA
    digest_algorithm: str = TOPOLOGY_CATALOG_DIGEST_ALGORITHM
    digest: str = ""
```

Array shapes

Let

- F be the number of selected frames;
- S be the number of source connectivity states;
- K be the number of stored topology records.

Then:

Field	dtype	shape
`frame_indices`	`int64`	`(F,)`
`frame_ids`	`int64`	`(F,)`
`frame_connectivity_state_ids`	`int32`	`(F,)`
`connectivity_state_topology_ids`	`int32`	`(S,)` in catalog mode
`frame_topology_ids`	`int32`	`(F,)`

In per-frame mode, `connectivity_state_topology_ids` must be an empty read-only array because a source state has no single public topology ID when it occurs in several frames. Internal memoization remains private.

Core result constraints

- All frame arrays are nonempty and have equal length.
- `frame_indices` and `frame_ids` match the source connectivity result exactly.
- Every frame connectivity state ID is valid for the source result.
- Every frame topology ID is valid for `topologies`.
- Catalog mode has one topology per exact class.
- Per-frame mode has $K = F$ and `frame_topology_ids == arange(F)`.
- `UNIFORM` requires catalog mode and $K = 1$.
- `PARTITIONED` requires catalog mode and $K \geq 2$.
- `PER_FRAME` requires per-frame mode.
- `UNDEFINED` is invalid in a constructed catalog.
- Topology IDs are deterministic and dense.
- Topologies are ordered by first occurrence in selected result order, with exact equality checked against existing digest buckets.
- Frame groups partition all selected positions exactly once.
- Segment and transition presence agrees with frame semantics and mode.
- Mapping digest is identical across all stored topologies.
- `metadata["source_connectivity_state_digests"]` aligns with the source state-to-topology map.
- Each stored topology retains the digest of its first-occurring representative connectivity state; additional source-state provenance is stored in catalog metadata.
- Metadata is deeply copied and exposed read-only.

Deterministic topology ID rule

In catalog mode, topology IDs are assigned by first occurrence in selected frame order, not by digest lexical order. This preserves intuitive correspondence with the analyzed data while remaining deterministic for a fixed frame selection.

For an ensemble, changing stored frame order may change dense topology IDs but must not change:

- the set of topology digests;
- exact topology class membership;
- class counts;
- serialized topology objects.

Scientific code must compare topology digests or exact objects across reordered ensembles, not assume that dense integer IDs are globally persistent.

Recommended convenience interface

`TopologyCatalog` should expose:

```
@property
def n_frames(self) -> int: ...

@property
def n_topologies(self) -> int: ...

@property
def topology_counts(self) -> NDArray[np.int64]: ...

@property
def topology_probabilities(self) -> NDArray[np.float64]: ...

@property
def is_uniform(self) -> bool: ...

@property
def is_partitioned(self) -> bool: ...

def topology_for_frame(self, frame_index: int) -> FrameworkTopology: ...

def topology_id_for_frame(self, frame_index: int) -> int: ...

def frames_for_topology(self, topology_id: int) -> NDArray[np.int64]: ...

def result_positions_for_topology(
    self, topology_id: int
) -> NDArray[np.int64]: ...

def connectivity_states_for_topology(
    self, topology_id: int
) -> NDArray[np.int32]: ...

def compare_topologies(
    self, topology_a: int, topology_b: int
) -> dict[str, tuple[FrameworkEdgeKey, ...]]: ...

def to_networkx(self, topology_id: int = 0) -> Any: ...

def to_dict(self) -> dict[str, Any]: ...

@classmethod
def from_dict(cls, payload: Mapping[str, Any]) -> "TopologyCatalog": ...
```

`frames_for_topology()` returns source collection frame indices. `result_positions_for_topology()` returns positions inside the catalog result. The distinction must remain explicit.

Primary build function

```
def build_topology_catalog(
    collection: AtomisticFrameCollection,
```

```
    connectivity: AtomicConnectivityResult,
    mapping: FrameworkMapping,
    *,
    validation_rules: FrameworkValidationRules | None = None,
    projection_options: FrameworkProjectionOptions | None = None,
    catalog_options: TopologyCatalogOptions | None = None,
) -> TopologyCatalog:
    """Classify projected framework topologies across selected frames."""
```

Inputs

`collection`

An `AtomisticFrameCollection` that owns the canonical atom identities and source frame metadata.

The module reads:

- `n_frames`;
- `n_atoms`;
- atomic numbers and atom ordering;
- `frame_ids`;
- `frame_semantics`;
- periodic-dimensionality convention.

The module does not require Cartesian positions or cells for projection because the source connectivity states already contain canonical periodic edge data.

`connectivity`

An `AtomicConnectivityResult` evaluated from the same collection and atom identity convention.

It supplies:

- selected frame indices and frame IDs;
- source connectivity state IDs;
- unique `AtomicConnectivityState` objects;
- the connectivity definition and persistent scope;
- state-level canonical periodic edge data;
- source provenance.

`mapping`

One immutable `FrameworkMapping` used for every source connectivity state. Changing mapping rules between frames is forbidden because topology equality would be undefined.

`validation_rules`

Optional `FrameworkValidationRules` passed unchanged to every `build_framework_topology()` call.

Validation may report material-specific defects but must not alter projected topology identity.

`projection_options`

Optional `FrameworkProjectionOptions` controlling bounded linker-path traversal. The same options apply to every projected state.

`catalog_options`

Optional `TopologyCatalogOptions`. Defaults are used when omitted.

Output

Returns one immutable `TopologyCatalog`.

The function must be deterministic for fixed inputs, options, and source result order.

Input constraints

Collection compatibility

The collection must satisfy:

- fixed atom count across frames;
- fixed atom ordering and physical identity;
- fixed per-index atomic numbers;
- explicit `FrameSemantics.TRAJECTORY` or `FrameSemantics.ENSEMBLE`;
- valid frame IDs for every selected source frame.

Equal composition without equal atom identity is insufficient.

Connectivity-result compatibility

The connectivity result must satisfy:

- every `frame_indices` entry is in range for the collection;
- `frame_ids` equal `collection.frame_ids[frame_indices]` exactly;
- metadata frame semantics agrees with `collection.frame_semantics` when present;
- every connectivity state uses identical active atom indices;
- every connectivity state uses identical active atomic numbers;
- every connectivity state uses identical `pbc`;
- every state belongs to the same persistent connectivity scope;
- state IDs and state digests are internally valid;
- selected frames are unique.

For trajectories, selected frame indices must be strictly increasing. Sparse selection is permitted, but transitions then describe changes between adjacent selected frames, not necessarily adjacent source MD steps.

For ensembles, stored order has no physical meaning. Frame indices must still be unique to prevent duplicate statistical weight unless a future weighted API is introduced.

Mapping and projection compatibility

- `mapping` must resolve at least one framework vertex in every state.
- Mapping digest and projection options are fixed across all states.
- All mapping atom-index overrides must be valid for the source collection.
- Every projected topology must pass structural construction invariants.
- Validation failures remain attached to topologies unless the underlying framework projector raises a configured hard error.

Mode constraints

- Per-frame mode is valid for trajectories and ensembles.
- Catalog mode is valid for trajectories and ensembles.
- Temporal segment persistence is evaluated only for trajectory catalog mode.
- The module must never infer transitions from ensemble frame adjacency.

Core algorithm

Step 1: validate source compatibility

Validate the collection, connectivity result, fixed scope, atom identity, selected frames, mapping, and options before projection begins.

Failure must occur before any partial catalog is returned.

Step 2: project source connectivity states

In catalog mode, call

```
build_framework_topology(
    state,
```

```
    mapping,
    validation_rules=validation_rules,
    options=projection_options,
)
```

once for each unique source connectivity state that is referenced by at least one selected frame.

Unused states in an externally constructed connectivity result should be rejected or ignored deterministically. The first implementation should reject them because state IDs are expected to form a compact source catalog.

In per-frame mode, the implementation may memoize projected states privately, but it must create one public topology record per selected frame.

Step 3: exact topology deduplication

For each projected state in first selected-frame occurrence order:

1. use `topology.digest` to select a candidate bucket;
2. construct or retrieve the canonical structural key $K(T)$;
3. compare $K(T)$ against every representative key already in that bucket;
4. reuse the matching topology ID if exact structured-key equality holds;
5. otherwise append a new topology class;
6. record the source connectivity-state to topology mapping.

The implementation must not trust a digest match without exact structured-key comparison. It must not use generic dataclass equality if that equality includes nonstructural `FrameworkEdgePath` provenance.

Step 4: assign frame topology IDs

For catalog mode,

$$t_p = q_{c_p}.$$

For per-frame mode,

$$t_p = p.$$

Step 5: classify consistency

```
mode == per_frame                  -> PER_FRAME
mode == catalog and K == 1         -> UNIFORM
mode == catalog and K >= 2         -> PARTITIONED
```

`UNDEFINED` is never emitted.

Step 6: construct frame groups

For every topology ID k, collect

$$G_k = \{p : t_p = k\}.$$

Store sorted result positions in one `TopologyFrameGroup`.

Step 7: construct trajectory segments

For trajectory catalog mode, run-length encode `frame_topology_ids` into maximal segments. Label each segment using `minimum_persistent_frames` without changing the assignment.

For ensembles or per-frame mode, set `segments=None`.

Step 8: construct transitions

For every adjacent pair of trajectory segments:

1. identify the last source position and first target position;
2. identify boundary connectivity state IDs;
3. compute atomic-edge set differences from the two source states;

4. compute framework-edge set differences from the two topologies;
5. derive affected atom, vertex, and linker sets;
6. record source and target frame metadata;
7. append one deterministic transition record.

Transitions are not constructed for ensembles.

Step 9: finalize identity and provenance

Create deeply immutable arrays and mappings. Compute the catalog digest from a canonical payload containing:

- schema version;
- frame semantics;
- consistency;
- mapping digest;
- projection and catalog options;
- selected frame indices and IDs;
- source connectivity state digests;
- stored topology digests;
- frame topology assignments;
- segments and transitions excluding nonidentity metadata.

Pseudocode

```
function build_topology_catalog(collection, connectivity, mapping, ...):
    validate_all_inputs()

    options = catalog_options or TopologyCatalogOptions()
    projection = projection_options or FrameworkProjectionOptions()

    if options.mode == "per_frame":
        memo = {}
        topologies = []
        for position in 0 .. F-1:
            state_id = connectivity.frame_state_ids[position]
            if state_id not in memo:
                memo[state_id] = build_framework_topology(...)
            topologies.append(copy_identity_reference(memo[state_id]))
        frame_topology_ids = arange(F)
        state_topology_ids = empty_int32()
        consistency = PER_FRAME
        groups = singleton_groups(F)
        segments = None
        transitions = ()
        return finalize(...)

    projected_by_state = {}
    for state_id in source_states_by_first_frame_occurrence:
        projected_by_state[state_id] = build_framework_topology(...)

    topology_classes = []
    digest_buckets = {}
    state_topology_ids = full(S, -1)

    for state_id in source_states_by_first_frame_occurrence:
        candidate = projected_by_state[state_id]
        candidate_key = framework_topology_structural_key(candidate)
        topology_id = exact_key_match_in_digest_bucket(
            candidate.digest,
            candidate_key,
            digest_buckets,
```

```
        )
        if topology_id is None:
            topology_id = append_new_class(candidate, candidate_key)
        state_topology_ids[state_id] = topology_id

    frame_topology_ids = state_topology_ids[frame_state_ids]
    consistency = UNIFORM if n_classes == 1 else PARTITIONED
    groups = build_frame_groups(frame_topology_ids)

    if collection.is_trajectory:
        segments = run_length_encode(frame_topology_ids)
        label_segment_persistence(segments, options.minimum_persistent_frames)
        transitions = build_exact_transitions(...)
    else:
        segments = None
        transitions = ()

    return finalize_immutable_catalog(...)
```

Exact topology comparison

Two `FrameworkTopology` objects belong to the same class only when their Stage 2 canonical structural keys are equal. Exact equality consists of:

- canonical framework-topology schema version;
- fixed mapping digest;
- retained vertex atom indices and aligned atomic numbers;
- periodic-dimensionality flags;
- sorted canonical `FrameworkEdgeKey` records, including endpoint identities, normalized periodic translations, ordered linker identities and image offsets, and whole-path rule IDs.

The implementation may use `graph_digest` and mapping-aware `digest` to locate a small candidate bucket, but it must verify the structured fields above before reusing a class ID.

The following do **not** define topology class identity:

- `source_connectivity_digest`;
- complete `FrameworkEdgePath` reconstruction fields that are not present in the canonical edge key;
- `OrientedFrameworkEdgePath` traversal orientation;
- projection reports and validation findings;
- degree and component arrays beyond invariant checking;
- Cartesian coordinates, cell shape, or frame index.

Two topologies with the same canonical graph but different validation warnings remain the same structural class, although provenance must preserve each projected state's diagnostic context.

Because one canonical topology object is retained per class, class-level topology validation should use the first projected representative. The catalog metadata should additionally map each source connectivity state to its projection validation summary so state-specific warnings are not lost.

Topology classes versus connectivity states

The mapping from connectivity states to topologies need not be one-to-one.

Many connectivity states to one topology

This occurs when changed atomic edges do not alter accepted projected framework paths, for example:

- spectator contacts change;
- excluded-atom contacts change;
- unused linker contacts change;
- different atomic wrapping normalizes to the same periodic graph;

- chemically irrelevant edges change outside the framework mapping.

The catalog must preserve

`connectivity_state_topology_ids`

so this compression remains inspectable.

One connectivity state to one topology

A deterministic fixed mapping always maps one canonical connectivity state to one canonical topology. One state must never map to several topology classes in catalog mode.

Mapping dependence

Topology class identity is mapping-dependent:

$$C_a \underset{M_1}{\sim} C_b \nRightarrow C_a \underset{M_2}{\sim} C_b.$$

Catalogs produced with different mapping digests must not be merged as if they shared one identity system.

Trajectory semantics

For a trajectory with assignments

`A A A B B A A`

there are two topology classes but three segments:

`A | B | A`

The result stores both:

- class-level frame groups for `A` and `B`;
- ordered segments 0, 1, and 2;
- transitions `A -> B` and `B -> A`.

A recurring topology reuses the same topology ID. Reappearance does not create a new class.

If selected frames are sparse, a transition means:

> the exact topology differs between two adjacent selected frames.

It does not prove the precise source MD step at which the event occurred.

Ensemble semantics

For an ensemble, the module constructs exact frame groups but no segments or transitions.

For assignments stored as

`A B A B`

and a reordered representation

`B A B A`

both results must contain the same exact topology objects and class counts. Dense topology IDs may differ because they follow first occurrence, but no scientific conclusion may depend on adjacent ensemble frame indices.

The module must never apply persistence thresholds, hysteresis, run-length confirmation, or transition timing to an ensemble.

Per-frame mode

Per-frame mode is an explicit escape hatch for collections where topology class reconciliation is unwanted or scientifically inappropriate.

It guarantees:

- one public topology entry per selected frame;
- `TopologyConsistency.PER_FRAME`;
- singleton frame groups;
- no segments;
- no transitions;
- no persistent topology identity claim.

The implementation may reuse internal computation for repeated connectivity states, but this optimization must not change the public contract.

Primitive-ring workflows should normally require catalog mode. A ring catalog can be built once per exact topology class. Per-frame mode implies one independent ring search per frame unless a later caller performs its own exact reconciliation.

Structural differences and affected atoms

Framework-edge comparison uses exact version-2 `FrameworkEdgeKey` records. The key stores the ordered linker atom identities relative to canonical endpoint orientation. Transition records store canonical undirected keys only; they do not store a separate edge for reverse traversal and do not require an orientation sign. A changed edge may reflect:

- changed endpoint identity;
- changed periodic translation;
- changed internal linker identity;
- changed linker image offsets;
- changed mapping rule ID.

Two edges with the same projected endpoint pair but different ordered linker path, whole-path rule, or periodic shift are distinct.

Affected sets are derived as follows:

$$A_{\mathrm{atomic}} = \bigcup_{e \in \Delta E_{\mathrm{atomic}}} \mathrm{endpoints}(e),$$

$$A_{\mathrm{vertex}} = \bigcup_{e \in \Delta E_{\mathrm{framework}}} \mathrm{projected\_endpoints}(e),$$

$$A_{\mathrm{linker}} = \bigcup_{e \in \Delta E_{\mathrm{framework}}} \mathrm{internal\_linkers}(e).$$

The complete affected atom set is

$$A = A_{\mathrm{atomic}} \cup A_{\mathrm{vertex}} \cup A_{\mathrm{linker}}.$$

These sets identify where topology changed. They do not assign a chemical event label such as bond breaking, dissolution, or defect formation.

Provenance and serialization

Every catalog must retain enough information to reproduce the classification:

- source collection frame semantics;
- selected collection frame indices and frame IDs;
- source atomic-connectivity definition and scope provenance;
- source connectivity state digests;
- connectivity-state to topology mapping;
- framework mapping and digest;
- framework projection options;
- framework validation rules;
- topology catalog options;
- topology schema versions and digests;

- frame topology IDs;
- segment persistence threshold;
- transition-difference storage policy;
- package version;
- implementation algorithm name.

Recommended metadata keys include:

```
algorithm = "exact_projected_topology_catalog"
projection_count
source_connectivity_state_count
stored_topology_count
compression_ratio_frames_to_topologies
compression_ratio_states_to_topologies
frame_semantics
transition_count
transient_segment_count
```

Serialization must use JSON-safe deterministic dictionaries and lists. `from_dict()` must recompute and verify every identity digest rather than trusting stored values.

Digest identity

The catalog digest identifies one classified result, including selected frame order and assignments. It is not a global topology-class identifier.

Global structural identity remains the individual `FrameworkTopology.digest`.

Therefore:

- same topologies in a different ensemble order may produce a different catalog digest;
- the topology digest set remains invariant;
- a different frame selection produces a different catalog digest;
- a different persistence threshold produces a different catalog digest even though frame topology assignments are unchanged;
- metadata such as wall time must not enter the digest.

Downstream category-consumer contract

A downstream consumer may condition an average or visualization on `frame_groups`, but must not reinterpret topology identity. For selected frames S, category k uses

$$S_k = S \cap G_k, \qquad p_k = \frac{|S_k|}{\sum_j |S_j|}.$$

Consumers must preserve:

- the catalog topology ID and exact `FrameworkTopology` object;
- frame membership and segment provenance;
- deterministic dominant-category selection by population, then topology ID;
- disjoint category frame sets;
- probabilities summing to one over the selected frames.

Category-conditioned averaging belongs to the consumer module. In particular, `mdstats.plotting.framework_dynamics` owns averaged framework geometry, atomic mean-connectivity layers, visibility defaults, and legend grouping. The catalog module does not create coordinates or Plotly traces.

Error model

```
class TopologyCatalogError(ValueError): ...
class TopologyCatalogInputError(TopologyCatalogError): ...
class TopologyCatalogProjectionError(TopologyCatalogError): ...
```

```
class TopologyCatalogConsistencyError(TopologyCatalogError): ...
class TopologyCatalogSerializationError(TopologyCatalogError): ...
```

Required failures

Raise `TopologyCatalogInputError` for:

- collection and connectivity frame-ID mismatch;
- inconsistent frame semantics;
- invalid or duplicate selected frames;
- nonmonotonic selected trajectory frames;
- inconsistent active atom populations across states;
- inconsistent PBC conventions;
- invalid mapping atom-index overrides;
- unsupported options.

Raise `TopologyCatalogProjectionError` when framework projection fails for a specific connectivity state. The error message must include the source state ID and digest, and the original framework exception must remain as the exception cause.

Raise `TopologyCatalogConsistencyError` for internally inconsistent topology IDs, frame groups, segments, transitions, or difference records.

Raise `TopologyCatalogSerializationError` when a serialized payload is missing required fields, carries an unsupported schema, or fails digest validation.

The module must not return a partial catalog after any error.

Complexity and performance

Let:

- F be selected frame count;
- S be unique source connectivity-state count;
- K be unique topology-class count;
- P_s be framework projection cost for state s;
- E_s be atomic edge count;
- E_k^F be projected framework edge count;
- Q be trajectory transition count.

Catalog mode projection cost is

$$O\left(\sum_{s=1}^{S} P_s\right),$$

not

$$O\left(\sum_{p=1}^{F} P_{c_p}\right).$$

Frame assignment, groups, and segment construction cost

$$O(F).$$

Average digest-bucket deduplication is approximately

$$O(S),$$

plus exact comparisons within digest buckets. Collision handling must remain correct even if a bucket contains several unequal topologies.

Transition-difference cost is approximately

$$\left(O\sum_{q=1}^{Q}(E_{a_q}+E_{b_q}+E^F_{A_q}+E^F_{B_q})\right).$$

Catalog-mode memory is approximately

$$O(F+S+K\bar{G}+Q\bar{D}),$$

where $\bar{G}$ is average stored topology size and $\bar{D}$ is average transition difference size.

Per-frame mode may require

$$O(F\bar{G})$$

public storage and should be used deliberately.

Edge cases and warnings

Different atomic ordering

Two files with the same composition but different atom ordering are not compatible without an explicit atom-identity mapping. The module must fail rather than infer identity from coordinates.

Same topology from different connectivity states

This is expected and must be compressed. Spectator or excluded-edge changes must not force a new framework topology class.

Validation report differences

The same exact topology may arise with different source-state projection warnings. Structural identity remains one class. State-specific diagnostics must remain accessible through metadata.

Digest collision

A digest collision must not merge unequal topologies. Exact structured equality is mandatory after digest bucketing.

Sparse trajectory selection

A transition between selected positions may span many omitted source frames. Transition records must report both selected result positions and collection frame indices. They must not claim an exact event time inside the gap.

Short transient segments

A persistence threshold labels short runs but does not remove them. Downstream analysis may filter confirmed segments, but raw topology evidence remains intact.

Rapid topology changes

A trajectory that changes topology almost every frame may produce $Q \approx F-1$. Catalog mode can still compress recurring classes, but transition storage may be large. Difference flags may be disabled when memory is more important than edge provenance.

Per-frame connectivity input

An `AtomicConnectivityResult` with `ConnectivityConsistency.PER_FRAME` is valid. Catalog mode may still deduplicate its projected topologies exactly.

Disconnected and broken frameworks

Disconnected topologies are valid unless framework validation rules reject them. The catalog must not assume one connected component.

Mapping changes

One catalog cannot contain several mappings. A new mapping produces a new catalog and a new topology identity system.

Cell deformation and rotation

Framework topology depends on canonical periodic edge records, not Cartesian cell shape. Compatible deformation or rigid rotation that preserves connectivity must not create a new topology class.

Asymmetric bridge orientation

Two connectivity states that project to `A-O-S-B` and `B-S-O-A` contain the same canonical decorated edge and therefore reconcile to one topology class when all other structural records match. A state projecting to `A-S-O-B` contains a different ordered linker path and must remain a different topology class.

Catalog code must compare canonical decorated edge keys, never only unordered projected endpoint pairs and never the direction in which a path happened to be discovered.

Atomic edge change without framework change

No topology transition is emitted when adjacent trajectory frames use different connectivity states that project to the same topology. The state change remains available in the source connectivity result.

Framework change without simple endpoint change

A path may change linker identity or periodic translation while retaining the same projected endpoint pair. Exact decorated edge comparison must detect the change.

Ensemble order

Frame adjacency in an ensemble has no physical meaning. The module creates no segments or transitions, even if stored topology IDs alternate.

Interaction with primitive-ring analysis

`primitive_ring.py` should consume one exact `FrameworkTopology` from the catalog at a time.

For a uniform catalog:

```
one topology -> one primitive-ring catalog
```

For a partitioned catalog:

```
K topology classes -> at most K primitive-ring catalogs
```

Exact ring identities may later be reconciled across topology classes using ring canonical keys. That reconciliation does not belong to Stage 3.

Per-frame mode provides no cross-frame topology-class promise and should normally produce independent ring catalogs.

Usage examples

Uniform trajectory

```python
from mdstats.analysis import (
    TopologyCatalogOptions,
    build_topology_catalog,
)

catalog = build_topology_catalog(
    collection,
    connectivity,
    mapping,
    validation_rules=validation_rules,
    catalog_options=TopologyCatalogOptions(mode="catalog"),
)

assert catalog.consistency.value == "uniform"
assert catalog.n_topologies == 1
```

Partitioned trajectory with descriptive persistence

```
catalog = build_topology_catalog(
    collection,
    connectivity,
    mapping,
    catalog_options=TopologyCatalogOptions(
        mode="catalog",
        minimum_persistent_frames=3,
    ),
)

for segment in catalog.segments or ():
    print(segment.topology_id, segment.status.value)

for transition in catalog.transitions:
    print(
        transition.source_topology_id,
        "->",
        transition.target_topology_id,
        transition.affected_atom_indices,
    )
```

Multi-topology ensemble

```
catalog = build_topology_catalog(
    ensemble,
    connectivity,
    mapping,
)

assert catalog.segments is None
assert catalog.transitions == ()

for group in catalog.frame_groups:
    print(group.topology_id, group.result_positions)
```

Explicit per-frame mode

```
catalog = build_topology_catalog(
    collection,
    connectivity,
    mapping,
    catalog_options=TopologyCatalogOptions(mode="per_frame"),
)

assert catalog.consistency.value == "per_frame"
assert catalog.n_topologies == catalog.n_frames
```

Implementation organization

The implementation should proceed in six controlled phases.

S3.1 - API and validation foundation

Implement:

- exception hierarchy;
- schema constants;
- `TopologyConsistency`;
- `TopologySegmentStatus`;
- `TopologyCatalogOptions`;
- `TopologyFrameGroup`;

- `TopologySegment`;
- input compatibility validation helpers.

Acceptance criteria:

- all value objects are immutable;
- invalid frame semantics or identity mismatches fail before projection;
- arrays use canonical dtypes and are read-only;
- option serialization is deterministic.

S3.2 - unique-state projection — implemented

Implement:

- first-occurrence ordering of referenced connectivity states;
- one `build_framework_topology()` call per referenced state in catalog mode;
- state-ID contextual error wrapping;
- state projection diagnostics;
- fixed mapping, validation, and projection options across states.

Acceptance criteria:

- repeated frames do not repeat projection;
- no coordinate-based bond reconstruction occurs;
- projection failure returns no partial catalog;
- source state digest is preserved by each topology.

S3.3 - exact topology reconciliation — implemented

Implement:

- digest buckets;
- a private canonical structural-key helper based on schema, mapping, vertices, PBC, and sorted `FrameworkEdgeKey` records;
- exact structured-key equality after digest lookup;
- deterministic topology IDs;
- connectivity-state to topology mapping;
- frame topology IDs;
- `UNIFORM`, `PARTITIONED`, and `PER_FRAME` classification;
- frame groups.

Acceptance criteria:

- different spectator connectivity states may map to one topology;
- unequal topologies with an artificial common digest remain separate;
- recurring $A - B - A$ assignments reuse topology ID `A`;
- ensemble reorder preserves topology digest set and class counts.

S3.4 - trajectory segments and transitions — implemented

Implement:

- run-length encoded segments;
- descriptive persistence status;
- exact atomic-edge differences;
- exact decorated framework-edge differences;
- affected atom, vertex, and linker sets;
- deterministic transition IDs;
- ensemble prohibition of segments and transitions.

Acceptance criteria:

- segments are maximal and cover all selected positions;

- short segments are labeled but not removed;
- transitions retain source and target frame metadata;
- sparse selections do not claim omitted event timing;
- no topology transition is emitted for atomic-state changes that project to one topology.

S3.5 - result object, serialization, and interoperability — implemented

Implement:

- `TopologyCatalog` invariants;
- convenience methods;
- dictionary round trips;
- catalog digest calculation and verification;
- NetworkX delegation to stored topology classes;
- public package exports;
- API docstrings and examples.

Acceptance criteria:

- round trips preserve all exact identities;
- malformed IDs, groups, segments, or transitions fail;
- metadata is deeply immutable;
- digest recomputation detects payload modification.

S3.6 - integration, documentation, and release — implemented

Complete:

- synthetic topology-class tests;
- trajectory and ensemble integration tests;
- Na-LTA uniform-topology acceptance;
- controlled bond-breaking partition test;
- full package regression suite;
- Markdown and PDF specification synchronization;
- source/specification consistency audit;
- version and changelog update when implementation begins.

Test specification

Input validation

1. collection and connectivity frame-ID agreement;
2. frame-semantics agreement;
3. duplicate selected-frame rejection;
4. nonmonotonic trajectory-frame rejection;
5. inconsistent active atom scope rejection;
6. inconsistent atomic-number identity rejection;
7. inconsistent PBC rejection;
8. invalid mapping override rejection;
9. invalid catalog options.

Projection reuse

10. one projection for one uniform source state across many frames;
11. one projection per unique referenced connectivity state;
12. unused source state handling;
13. contextual projection error with state ID and digest;
14. fixed projection options across all states.

Exact class reconciliation

15. uniform trajectory;
16. uniform ensemble;
17. two distinct topology classes;
18. recurring $A - B - A$ trajectory;
19. several connectivity states projecting to one topology;
20. digest-collision guard using exact structured-key equality;
21. deterministic first-occurrence topology IDs;
22. ensemble reorder invariance of topology digest set and counts;
23. `A-O-S-B` and reverse traversal `B-S-O-A` reconcile to one class;
24. `A-O-S-B` and `A-S-O-B` remain distinct classes;
25. reverse discovery of one edge creates no false remove/add transition;
26. self-image edge representations $\mathbf{M}$ and $-\mathbf{M}$ reconcile after Stage 2 canonicalization when such edges are supplied by a compatible Stage 2 topology fixture; the current atomic-connectivity builder does not emit self-image atomic edges, so this remains a direct structural-key test rather than an end-to-end connectivity test;
27. nonidentity validation or projection diagnostics do not split a class.

Frame groups and consistency

28. frame-group partition completeness;
29. frame-group sorted uniqueness;
30. correct `UNIFORM` classification;
31. correct `PARTITIONED` classification;
32. correct `PER_FRAME` classification;
33. rejection of `UNDEFINED` in constructed results.

Segments and persistence

34. one uniform trajectory segment;
35. maximal $A|B|A$ segmentation;
36. dense segment IDs;
37. confirmed segment at threshold;
38. transient segment below threshold;
39. threshold does not alter frame assignments;
40. no segments for ensembles;
41. no segments for per-frame mode.

Transitions

42. exact source and target boundary positions;
43. added and removed atomic edges;
44. added and removed decorated framework edges;
45. affected atom indices;
46. affected vertex indices;
47. affected linker indices;
48. transition into and out of a transient segment;
49. no transition for atomic-state changes with unchanged projected topology;
50. empty transitions for ensembles;
51. sparse trajectory selection metadata.

Persistence and serialization

52. deeply immutable arrays and mappings;
53. catalog dictionary round trip;
54. unsupported schema rejection;
55. modified payload digest rejection;
56. convenience-method frame/index distinction;

57. NetworkX conversion for selected topology.

Domain integration

58. relaxed Na-LTA trajectory remains one topology class;
59. broad Na-O connectivity states still project to the same LTA topology;
60. controlled framework-edge removal creates a second topology class;
61. disconnected framework state remains representable;
62. variable-cell but connectivity-preserving frames remain uniform;
63. complete package regression suite remains clean.

Implementation and validation status

The `mdstats 0.16.0` implementation provides the complete public API above. Its focused regression suite contains 26 executable Stage 3 tests covering the core input, identity, trajectory, ensemble, transition, serialization, and LTA-domain contracts. The broader numbered matrix below remains normative: some cases are covered transitively by existing Stage 1/2 tests or require future fixtures such as externally supplied self-image projected edges. The complete package suite passes 322 tests with 27 expected scientific or visualization warnings.

Acceptance criteria for Stage 3

Stage 3 is accepted when:

1. the public API matches this specification;
2. all identity-bearing objects are immutable;
3. collection and connectivity compatibility is checked explicitly;
4. each referenced connectivity state is projected at most once in catalog mode;
5. topology classes use exact Stage 2 canonical structural-key equality;
6. reverse traversal never creates a second edge or topology class;
7. asymmetric linker order remains structurally distinguishable;
8. digest collisions cannot merge unequal topologies;
9. dense topology IDs are deterministic;
10. frame groups are valid for trajectories and ensembles;
11. trajectory segments are maximal and ordered;
12. ensembles never receive temporal transitions;
13. transient segments are labeled without hidden smoothing;
14. transitions retain exact atomic and decorated framework-edge differences;
15. recurring topology classes reuse identity;
16. same-topology connectivity changes do not create false topology transitions;
17. serialization round trips verify all digests;
18. Na-LTA uniform and controlled-defect tests pass;
19. Markdown and PDF specifications match the implemented source;
20. the complete mdstats regression suite remains clean.

Deferred functionality

The Stage 3 implementation intentionally defers:

- automatic topology-state smoothing or segment merging;
- probabilistic topology classes;
- approximate graph matching;
- graph isomorphism across different atom orderings;
- automatic atom-identity mapping;
- equivalence across primitive cells, supercells, or lattice bases;
- inferred topology lineage when exact keys differ;
- incremental local framework projection after one edge change;
- persistent disk caches across processes;

- weighted ensemble frames;
- topology transition kinetics;
- primitive-ring enumeration;
- global ring identity across topology classes;
- dynamic region membership and detachment classification;
- automatic chemical event naming.

Deferral is deliberate. These features require separate scientific definitions and testable contracts.

AI implementation context

An implementation agent should preserve the following boundaries:

1. Do not call the geometric neighbor kernel from `topology_catalog.py`.
2. Treat `AtomicConnectivityResult` as the atomic-edge authority.
3. Treat `FrameworkMapping` as fixed scientific provenance.
4. Call `build_framework_topology()` only on unique source states in catalog mode.
5. Use topology digest only for bucketing; use the Stage 2 canonical structural key for class identity. Do not rely on generic `FrameworkTopology` or `FrameworkEdgePath` object equality if it includes nonstructural provenance.
6. Do not infer time order from frame array order when semantics are `ENSEMBLE`.
7. Do not merge or relabel short topology segments.
8. Do not create transitions for atomic-state changes that leave projected topology unchanged.
9. Preserve periodic edge shifts and internal linker identities in differences.
10. Keep topology classes, frame groups, trajectory segments, and transitions as separate data structures.
11. Do not start primitive-ring search inside this module.
12. Fail explicitly on incompatible atom identity or mapping provenance.

Final architectural invariant

The module must preserve the separation

$$\boxed{\text{atomic connectivity states} \rightarrow \text{exact framework topology classes} \rightarrow \text{trajectory segments or ensemble groups}}\,.$$

Topology class identity is structural and mapping-dependent. Trajectory segments are ordered observations of those classes. Ensemble groups are unordered samples. Neither concept replaces the other.

The next module after the completed Stage 3 implementation is

`primitive_ring.py`

which should enumerate canonical removed-edge shortest-path primitive rings once per exact `FrameworkTopology` class.